Team StockPulse

Jayden Williams

Amaya Laing

Cameron Smith

Bruce Webb

Real-Time Stock Monitoring Application

Our objective was to create a real-time stock monitoring application using Task Parallel Library for concurrent data fetching and LINQ for analysis.

The Challenge: Navigating Market Volatility in Real-Time

Immediate Data is Non-Negotiable

Stock prices shift in milliseconds. Delayed or inaccurate data leads to missed opportunities and costly decisions.

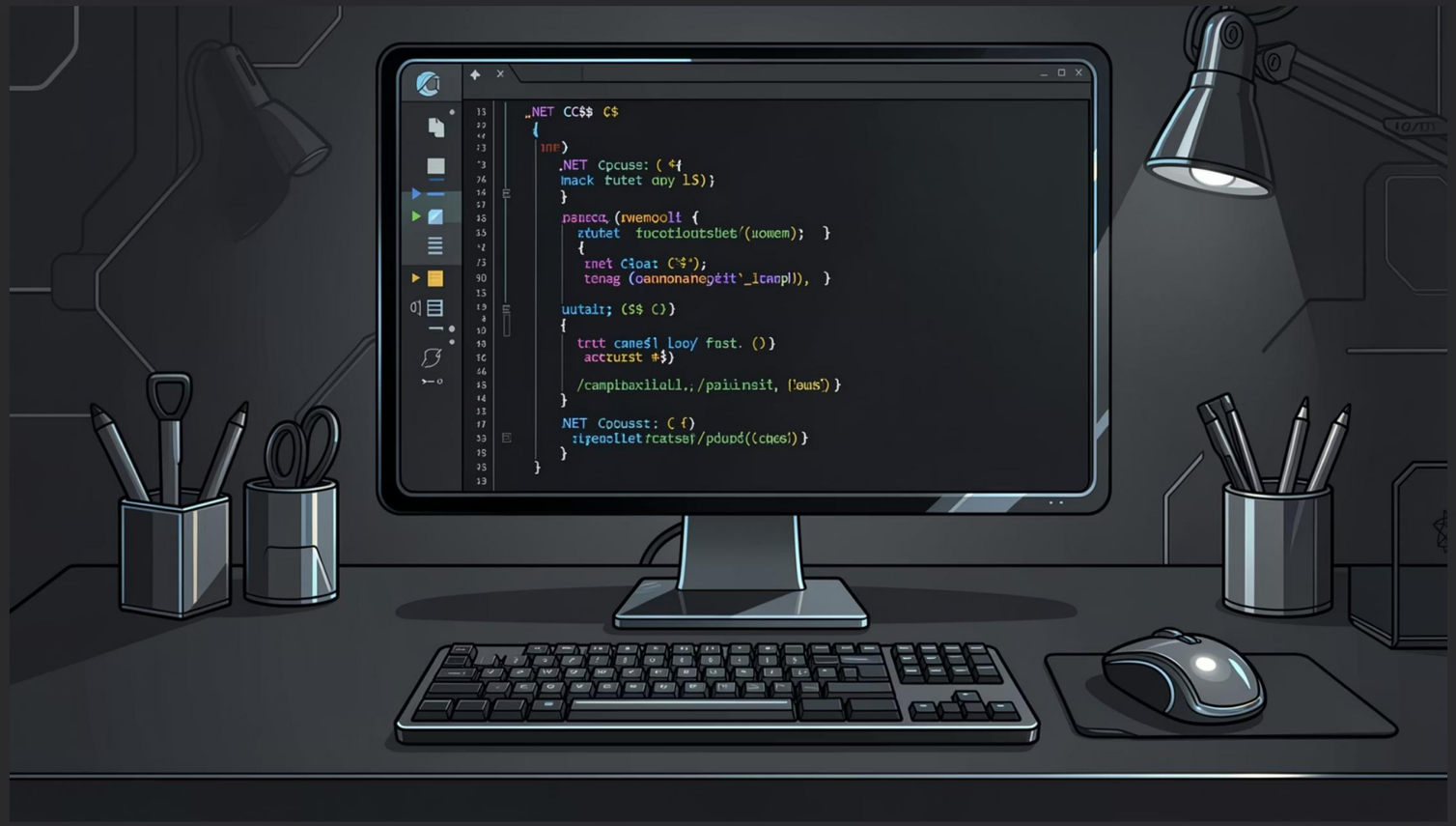
Traditional Methods Fall Short

Sequential data fetching can't keep pace with the volume and velocity of live market feeds.

Our Goal

Build a robust, efficient, and insightful monitoring tool that delivers real-time clarity.

Our Solution: Leveraging .NET Powerhouses



Task Parallel Library (TPL)

Orchestrates concurrent API calls, enabling simultaneous data fetching across multiple stock symbols no blocking, no bottlenecks.

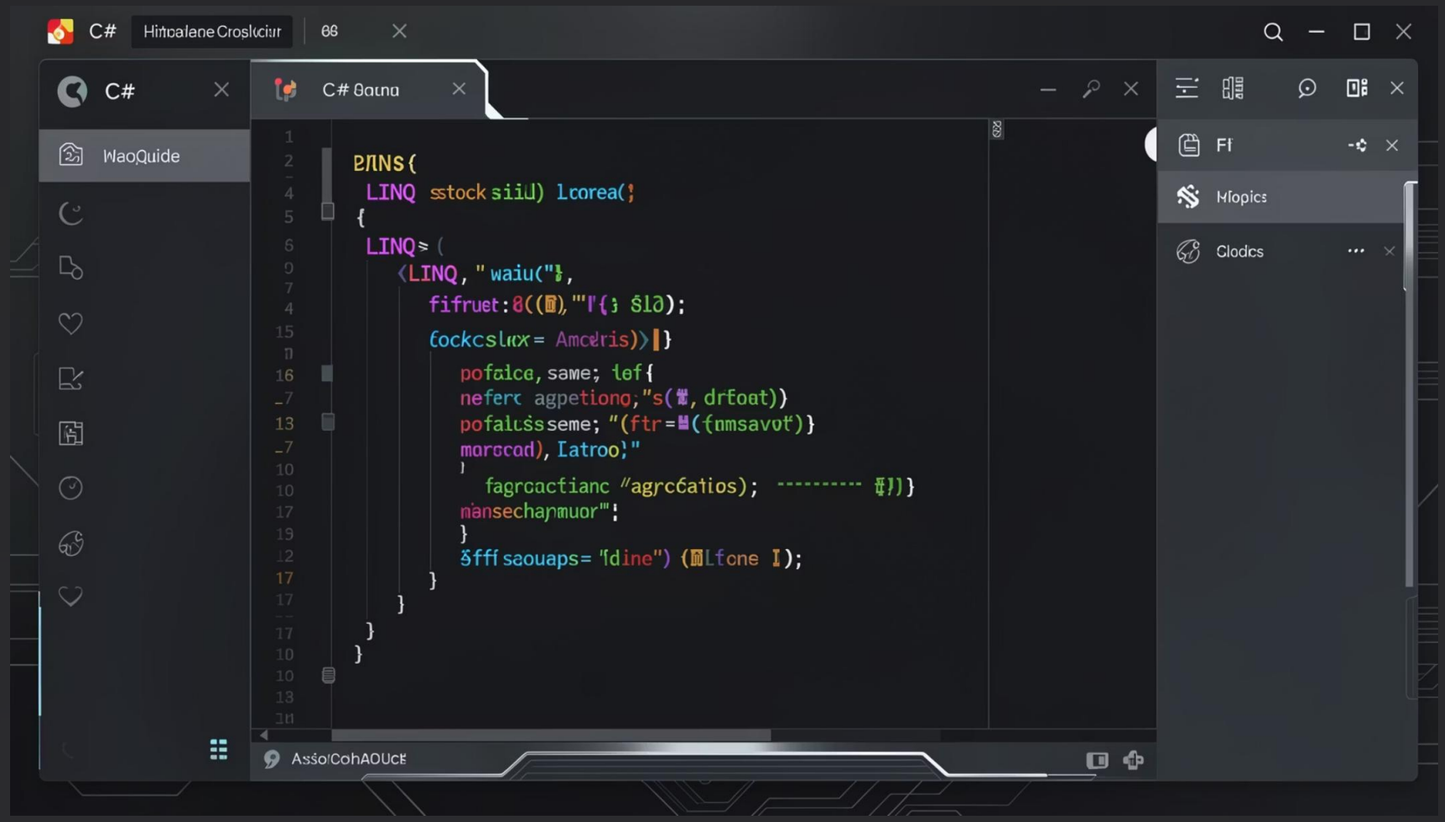
LINQ

Language Integrated Query transforms raw market data into actionable insights through expressive filtering, aggregation, and sorting.

Data Analysis with LINQ

Powerful Query Capabilities

LINQ enables complex analytical operations directly in C# this allows clean, readable, and performant.



Filtering

Isolate stocks by sector, volume, or price threshold.

Aggregation

Compute moving averages, gains, and portfolio totals.

Sorting

Rank symbols by performance, volatility, or custom metrics.

Filtering with Where

What it does

Filters the stocks array down to only those with a positive ChangePercent . Isolating gainers from the full dataset in a single readable expression.

Where it appears in the code:

After the stocks array is fetched. The resulting gainers list is used to print the 'Stocks Gaining: X/7' summary line at the bottom of the output.

FILTERING

From our code:

```
var gainers = stocks
    .Where(s => s.ChangePercent > 0)
    .ToList();

// Used in output:
Console.WriteLine(
    $"Stocks Gaining: {gainers.Count}/{stocks.Length}");
```

Before filter → After filter:

NVDA	+4.82%	✓ included in gainers
AAPL	+2.11%	✓ included in gainers
GOOG	-0.33%	✗ filtered out
META	-1.45%	✗ filtered out
TSLA	-3.97%	✗ filtered out

Selection with First

What it does

Plucks the single best or worst performing stock from the array by chaining .First() after an OrderBy or OrderByDescending call returning just one Stock object rather than a list.

Where it appears in the code:

Two separate LINQ chains run after the main table is printed. One sorts descending (top gainer), one ascending (top loser). Both printed as a highlighted summary below the table.

From our code:

```
var topGainer = stocks
    .OrderByDescending(s => s.ChangePercent)
    .First();

var topLoser = stocks
    .OrderBy(s => s.ChangePercent)
    .First();
```

TOP GAINER

NVDA

+4.82%

increased by 4.82%

TOP LOSER

TSLA

-3.97%

decreased by 3.97%

SELECTION

Aggregation with Average

What it does

Computes the mean price across all 7 monitored stocks in a single LINQ expression. There are no loops, no manual summation. Math.Round keeps the result clean to 2 decimal places.

Where it appears in the code:

Calculated right after the stocks array is fetched. Displayed as a portfolio-level summary at the bottom of each console refresh cycle.

From your code:

```
decimal avgPrice =
    Math.Round(
        stocks.Average(s => s.Price), 2);

Console.WriteLine($"Average Price: ${avgPrice}");
```

Sample Console Output

$284.63

Average price across all 7 symbols

AGGREGATION

Sorting with OrderByDescending

What it does

Sorts the entire array of fetched stocks from highest to lowest change percent using a single LINQ expression. No manual loops needed. The best-performing stocks always appear at the top of the console output.

Where it appears in the code:

After Task.WhenAll() resolves and stocks[] is populated. The full array is sorted before the table is printed to the console.

SORTING

From our code:

```
var sortedStocks = stocks
    .OrderByDescending(stock => stock.ChangePercent)
    .ToList();
```

Console output (sorted highest → lowest):

Symbol	Price	Change %	Last Updated
NVDA	$421.50	+4.82%	10:14:32 AM
AAPL	$198.34	+2.11%	10:14:33 AM
GOOG	$175.20	-0.33%	10:14:31 AM
TSLA	$87.22	-3.97%	10:14:34 AM

TPL IN ACTION

Task Parallel Library (TPL)

What is TPL?

The Task Parallel Library is a .NET framework that lets you run multiple operations concurrently using async/await and Task objects without blocking the main thread or writing complex threading code.

Why it matters for StockPulse

Without TPL, each of the 7 stocks would fetch one at a time up to 10.5 seconds total. With Task.WhenAll(), all 7 run simultaneously, cutting wait time down to the slowest single fetch (~1.5s max).

CONCURRENCY

How we used it — 3 key spots:

1 async Task<Stock> — the method signature

Returns a Task instead of a plain value, making it non-blocking.

```
public async Task<Stock> FetchStockDataAsync(string symbol)
```

2 await Task.Delay() — non-blocking wait

Simulates an API call delay without freezing the thread.

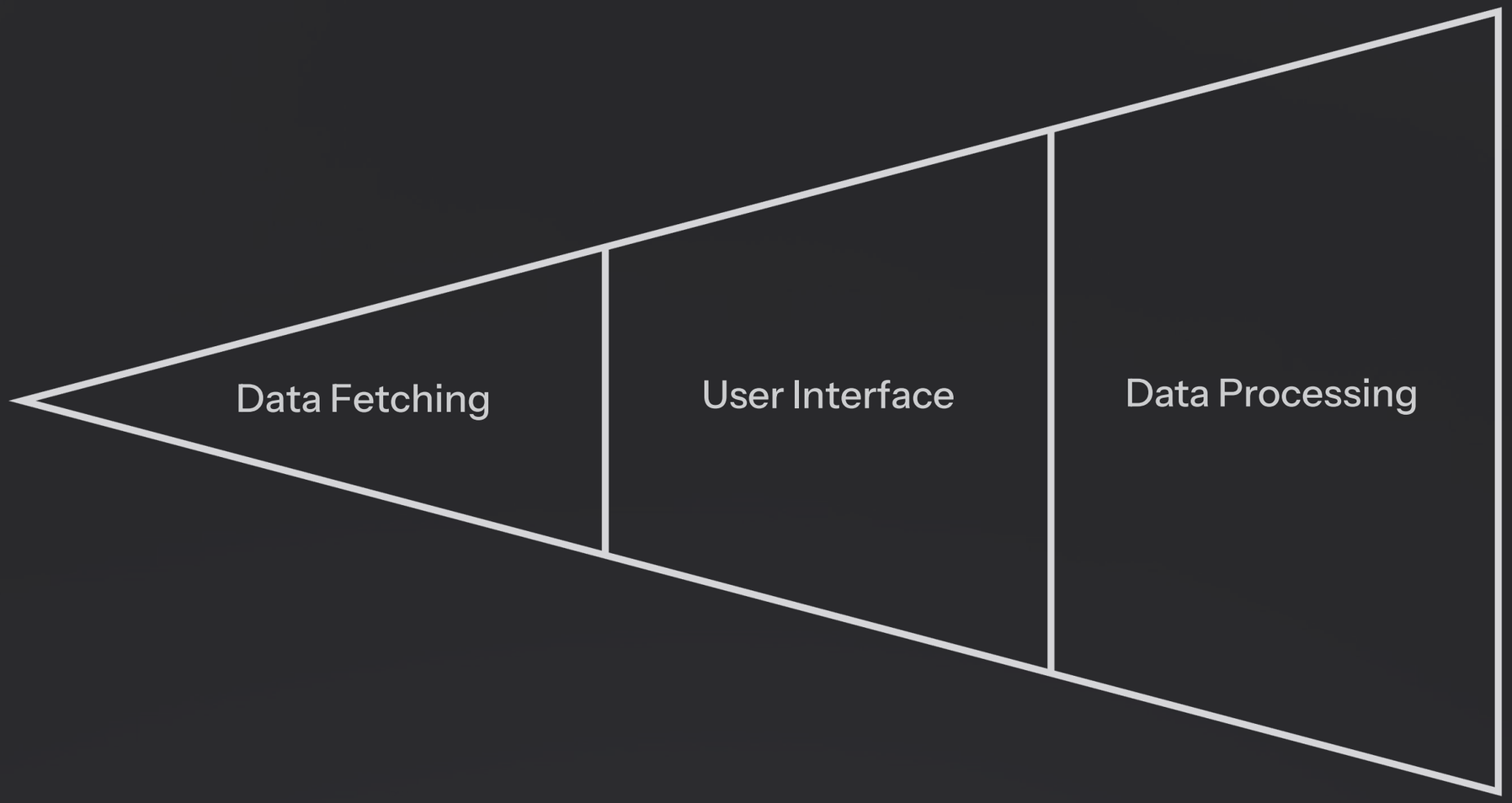
```
await Task.Delay(random.Next(500, 1500));
```

3 Task.WhenAll() — concurrent fetching

Launches all 7 fetches at the same time and waits for all to complete.

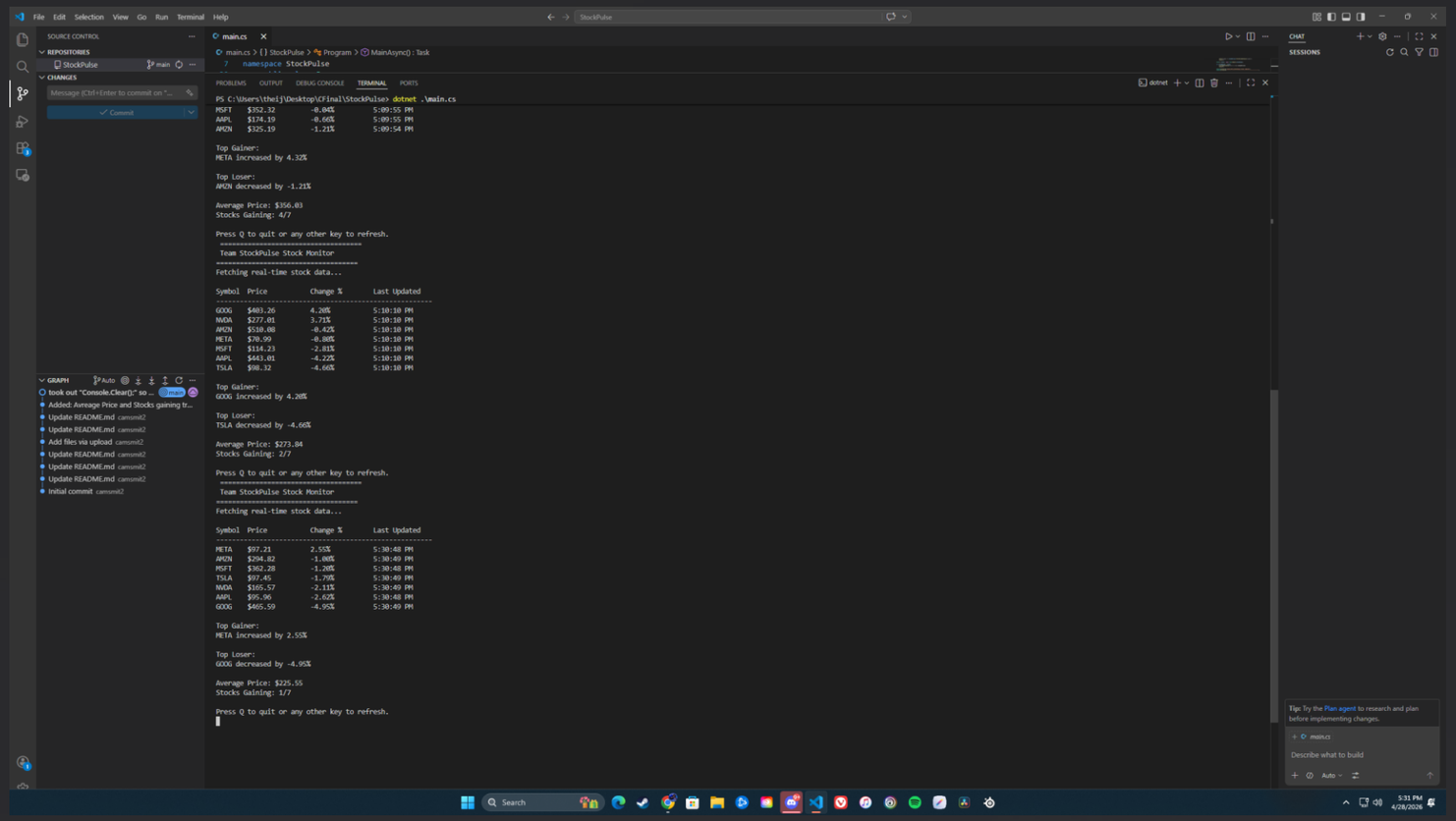
```
// Kick off all 7 fetch tasks simultaneously
List<Task<Stock>> stockTasks = stockSymbols
    .Select(symbol => FetchStockDataAsync(symbol))
    .ToList();

// Wait for ALL to finish, then get results
Stock[] stocks = await Task.WhenAll(stockTasks);
```

Architecture Deep Dive: How It Works



The architecture follows a clean pipeline: raw data enters through concurrent TPL tasks, is refined by LINQ expressions, and is surfaced to users through a responsive, real-time interface — all with minimal latency.

Application Output



Main Interface Overview

This is the main output of the program. It displays:

- The names of the stocks we are tracking
- The price of the stock
- The percentage and change of the stock whenever you refresh
- The time of the stock request
- The top gaining stock
- The top losing stock
- The average price between the stocks

Video Demonstration

That’s a Wrap

Our application shows the fast-paced stock market. Combining the concurrency of **TPL** with the analytical precision of **LINQ** to transform raw data into real-time clarity.

Built With	Core Strength
.NET · TPL · LINQ	Concurrency + Analysis

Thank You

Questions & Discussion Welcome